# KubiSat Ground

Generated by Doxygen 1.13.2

**1 pico-lora** 1
1.1 Supported Hardware . . . 1
1.1.1 Default Pinout . . . 1
1.2 Installation . . . 2
1.3 Notes . . . 2
1.4 References . . . 2

**2 File Index** 3
2.1 File List . . . 3

**3 File Documentation** 5
3.1 main.cpp File Reference . . . 5
3.1.1 Detailed Description . . . 6
3.1.2 Function Documentation . . . 6
3.1.2.1 onReceive() . . . 6
3.1.2.2 sendMessage() . . . 7
3.1.2.3 initSystems() . . . 8
3.1.2.4 main() . . . 9
3.1.3 Variable Documentation . . . 9
3.1.3.1 localAddress . . . 9
3.1.3.2 destination . . . 9
3.1.3.3 REPLY_TIMEOUT . . . 10
3.2 main.cpp . . . 10
3.3 pin_config.cpp File Reference . . . 12
3.3.1 Variable Documentation . . . 12
3.3.1.1 csPin . . . 12
3.3.1.2 resetPin . . . 12
3.3.1.3 irqPin . . . 12
3.4 pin_config.cpp . . . 13
3.5 pin_config.h File Reference . . . 13
3.5.1 Macro Definition Documentation . . . 14
3.5.1.1 DEBUG_UART_PORT . . . 14
3.5.1.2 BAUD_RATE . . . 14
3.5.1.3 UART_TX_PIN . . . 14
3.5.1.4 UART_RX_PIN . . . 15
3.5.1.5 PIN_MISO . . . 15
3.5.1.6 PIN_CS . . . 15
3.5.1.7 PIN_SCK . . . 15
3.5.1.8 PIN_MOSI . . . 15
3.5.1.9 SPI_PORT . . . 15
3.5.1.10 READ_BIT . . . 15
3.5.1.11 LORA_DEFAULT_SPI . . . 15
3.5.1.12 LORA_DEFAULT_SPI_FREQUENCY . . . 16

3.5.1.13 LORA_DEFAULT_SS_PIN . . . 16
3.5.1.14 LORA_DEFAULT_RESET_PIN . . . 16
3.5.1.15 LORA_DEFAULT_DIO0_PIN . . . 16
3.5.1.16 PA_OUTPUT_RFO_PIN . . . 16
3.5.1.17 PA_OUTPUT_PA_BOOST_PIN . . . 16
3.5.2 Variable Documentation . . . 16
3.5.2.1 csPin . . . 16
3.5.2.2 resetPin . . . 17
3.5.2.3 irqPin . . . 17
3.5.2.4 localAddress . . . 17
3.5.2.5 destination . . . 17
3.6 pin_config.h . . . 17
3.7 README.md File Reference . . . 18
3.8 utils.cpp File Reference . . . 18
3.8.1 Function Documentation . . . 18
3.8.1.1 uartPrint() . . . 18
3.9 utils.cpp . . . 19
3.10 utils.h File Reference . . . 19
3.10.1 Function Documentation . . . 20
3.10.1.1 uartPrint() . . . 20
3.11 utils.h . . . 21

**Index** **23**

# Chapter 1

# pico-lora

LoRa communication library for Raspberry Pi RP2040 Microcontroller.

This library is written for RP2040 C++ SDK.

## 1.1 Supported Hardware

- RP2040 boards
  - Raspberry Pi Pico
- Semtech SX1276/77/78/79 based LoRa boards

### 1.1.1 Default Pinout

| Raspberry Pi Pico | Semtech SX1278 |
|---|---|
| 3.3V | VCC |
| GND | GND |
| GPIO 18 | SCK |
| GPIO 19 | MOSI |
| GPIO 16 | MISO |
| GPIO 7 | DIO0 / G0 |
| GPIO 8 | NSS / CS |
| GPIO 9 | RESET |
| GPIO 10 | DIO1 / G1 |

Default Pinout can be overrided with setPins() function

## 1.2 Installation

1. Set up the Pico C/C++ SDK
2. Download module from git and open directory

```
git clone https://github.com/akshayabali/pico-lora.git
cd ./pico-lora
```

3. Set `PICO_SDK_PATH`

```
export PICO_SDK_PATH="path to pico-sdk"
```

4. Create `build` directory then run `cmake` and `make`:

```
mkdir build
cd build
cmake ..
make
```

## 1.3 Notes

Currently this is only tested on Raspberry Pi Pico and Semtech1278 board. Feel free to reach out for any bugs or support.

## 1.4 References

This `project` is based on LoRa Library for Arduino: `sandeepmistry/arduino-LoRa`

# Chapter 2

# File Index

## 2.1 File List

Here is a list of all files with brief descriptions:


main.cpp
KubiSat Ground Station Firmware main implementation . . . . . 5
pin_config.cpp . . . . . 12
pin_config.h . . . . . 13
utils.cpp . . . . . 18
utils.h . . . . . 19

# Chapter 3

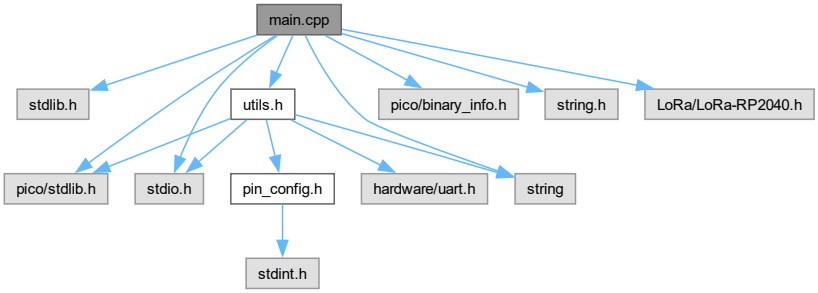
# File Documentation

## 3.1 main.cpp File Reference

KubiSat Ground Station Firmware main implementation.

```
#include "stdlib.h"
#include "pico/stdlib.h"
#include "pico/binary_info.h"
#include "stdio.h"
#include "string.h"
#include "string"
#include "utils.h"
#include "LoRa/LoRa-RP2040.h"
```

Include dependency graph for main.cpp:




### Functions

- void onReceive (int packetSize)

  *Handler for received LoRa packets.*
- void sendMessage (string outgoing)

  *Sends a message via LoRa radio.*
- void initSystems ()

  *Initializes all system peripherals.*
- int main ()

  *Main program entry point.*

**Variables**

- uint8_t localAddress = 21

  *Address of this device (ground station)*
- uint8_t destination = 37

  *Address of the destination device (satellite)*
- const uint32_t REPLY_TIMEOUT = 5000

  *Timeout for waiting for a reply in milliseconds.*

### 3.1.1 Detailed Description

KubiSat Ground Station Firmware main implementation.

**Author**

KubiSat Team

**Date**

2025

This file implements the ground station firmware for the KubiSat project. It handles LoRa radio communication with the satellite and provides a UART interface for control and monitoring from a PC.

Definition in file main.cpp.

### 3.1.2 Function Documentation

#### 3.1.2.1 onReceive()

```
void onReceive (
            int packetSize)
```

Handler for received LoRa packets.

This function processes incoming LoRa radio packets, verifies addressing, and checks for valid frame format before forwarding to UART.

**Parameters**

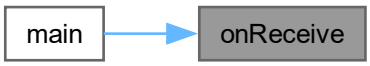
| | |
|---|---|
| *packetSize* | Size of the received packet in bytes |

Definition at line 47 of file main.cpp.

Here is the call graph for this function:




Here is the caller graph for this function:




#### 3.1.2.2 sendMessage()

```
void sendMessage (
            string outgoing)
```

Sends a message via LoRa radio.

This function formats and transmits a message through the LoRa radio, adding appropriate addressing and logging the transmission.

**Parameters**

| | |
|---|---|
| *outgoing* | The message payload to be sent |

Definition at line 92 of file main.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:




#### 3.1.2.3 initSystems()

```
void initSystems ()
```

Initializes all system peripherals.

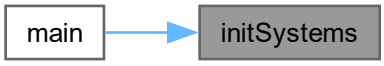
This function configures UART, initializes the LoRa radio module with the specified frequency, and prepares the system for operation.

Definition at line 116 of file main.cpp.

Here is the call graph for this function:




Here is the caller graph for this function:



#### 3.1.2.4 main()

```
int main ()
```

Main program entry point.

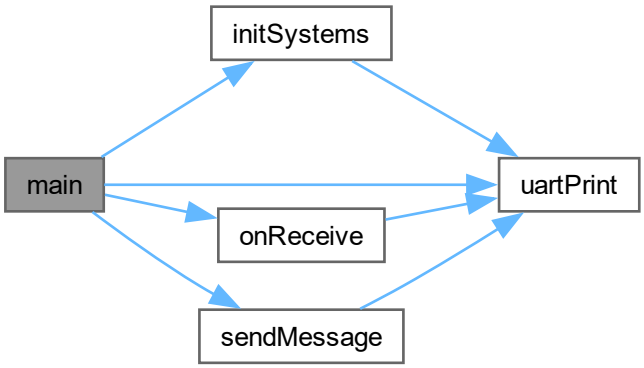
This function implements the main control flow of the ground station firmware, including the secure handshake protocol, connection maintenance, and data forwarding between UART and LoRa interfaces.

**Returns**

int Never returns (infinite loop)

Definition at line 147 of file main.cpp.

Here is the call graph for this function:




### 3.1.3 Variable Documentation

#### 3.1.3.1 localAddress

```
uint8_t localAddress = 21
```

Address of this device (ground station)

Definition at line 26 of file main.cpp.

#### 3.1.3.2 destination

```
uint8_t destination = 37
```

Address of the destination device (satellite)

Definition at line 31 of file main.cpp.

#### 3.1.3.3 REPLY_TIMEOUT

```
const uint32_t REPLY_TIMEOUT = 5000
```

Timeout for waiting for a reply in milliseconds.

Definition at line 36 of file main.cpp.

## 3.2 main.cpp

Go to the documentation of this file.

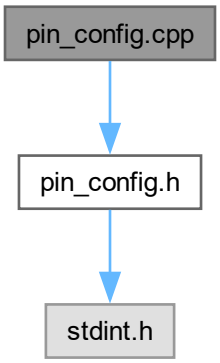
```
00001
00011
00012 #include "stdlib.h"
00013 #include "pico/stdlib.h"
00014 #include "pico/binary_info.h"
00015 #include "stdio.h"
00016 #include "string.h"
00017 #include "string"
00018 #include "utils.h"
00019 #include "LoRa/LoRa-RP2040.h"
00020
00021 using std::string;
00022
00026 uint8_t localAddress = 21;
00027
00031 uint8_t destination = 37;
00032
00036 const uint32_t REPLY_TIMEOUT = 5000;
00037
00038
00047 void onReceive(int packetSize) {
00048     if (packetSize == 0)
00049         return;
00050
00051     uint8_t buffer[256];
00052     int bytesRead = 0;
00053
00054     while (LoRa.available() && bytesRead < packetSize && bytesRead < 256) {
00055         buffer[bytesRead++] = LoRa.read();
00056     }
00057
00058     uint8_t receivedDestination = buffer[1];
00059     uint8_t receivedLocalAddress = buffer[0];
00060
00061     if (receivedDestination != destination) {
00062         uartPrint("Error: Destination address mismatch");
00063         return;
00064     }
00065
00066     if (receivedLocalAddress != localAddress) {
00067         uartPrint("Error: Local address mismatch!");
00068         return;
00069     }
00070
00071     std::string received(reinterpret_cast<char*>(buffer + 2), bytesRead - 2);
00072
00073     size_t headerPos = received.find("KBST");
00074     size_t footerPos = received.find(";TSBK");
00075
00076     if (headerPos != std::string::npos && footerPos != std::string::npos && footerPos > headerPos) {
00077         uartPrint("0x" + std::to_string(destination) + " ==> " + "0x" + std::to_string(localAddress) +
      " : " + received);
00078     } else {
00079         uartPrint("No valid frame found in received data");
00080     }
00081 }
00082
00083
00092 void sendMessage(string outgoing)
00093 {
00094     int n = outgoing.length();
00095     char send[n + 1];
00096     strcpy(send, outgoing.c_str());
00097
00098     LoRa.beginPacket();        // start packet
00099     LoRa.write(destination);  // add destination address
```

```
00100     LoRa.write(localAddress); // add sender address
00101     LoRa.print(send);         // add payload
00102     LoRa.endPacket(false);    // finish packet and send it
00103 
00104     uartPrint("0x" + std::to_string(destination) + " <== " + "0x" + std::to_string(localAddress) + " :
      " + send);
00105 
00106     LoRa.flush();
00107 }
00108 
00109 
00116 void initSystems() {
00117     stdio_init_all();
00118 
00119     uart_init(DEBUG_UART_PORT, BAUD_RATE);
00120     gpio_set_function(UART_TX_PIN, GPIO_FUNC_UART);
00121     gpio_set_function(UART_RX_PIN, GPIO_FUNC_UART);
00122     uart_set_hw_flow(DEBUG_UART_PORT, false, false);
00123     uart_set_format(DEBUG_UART_PORT, 8, 1, UART_PARITY_NONE);
00124 
00125     uartPrint("\nPower on");
00126 
00127     LoRa.setPins(csPin, resetPin, irqPin);
00128     long frequency = 433E6;
00129     if (!LoRa.begin(frequency)) {
00130         uartPrint("LoRa init failed. Check your connections.");
00131         while (true);
00132     }
00133 
00134     uartPrint("LoRa initialized with frequency " + std::to_string(frequency));
00135 }
00136 
00137 
00147 int main() {
00148     initSystems();
00149     uartPrint("Starting LoRa <=> UART routine.");
00150 
00151     string uartBuffer;
00152     while (1) {
00153 
00154         int packetSize = LoRa.parsePacket();
00155         if (packetSize)
00156         {
00157             onReceive(packetSize);
00158         }
00159 
00160         while (uart_is_readable(DEBUG_UART_PORT)) {
00161             char c = uart_getc(DEBUG_UART_PORT);
00162             if (c == '\n' || c == '\r') {
00163                 if (!uartBuffer.empty()) {
00164                     sendMessage(uartBuffer);
00165                     uartBuffer.clear();
00166                 }
00167             } else {
00168                 uartBuffer += c;
00169             }
00170         }
00171         sleep_ms(10);
00172     }
00173 }
```

## 3.3 pin_config.cpp File Reference

```
#include "pin_config.h"
```

Include dependency graph for pin_config.cpp:




### Variables

- const int csPin = 1
- const int resetPin = 29
- const int irqPin = 10

### 3.3.1 Variable Documentation

#### 3.3.1.1 csPin

```
const int csPin = 1
```

Definition at line 3 of file pin_config.cpp.

#### 3.3.1.2 resetPin

```
const int resetPin = 29
```

Definition at line 4 of file pin_config.cpp.

#### 3.3.1.3 irqPin

```
const int irqPin = 10
```

Definition at line 5 of file pin_config.cpp.

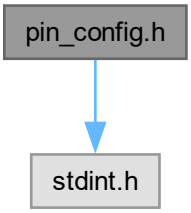
## 3.4 pin_config.cpp

Go to the documentation of this file.

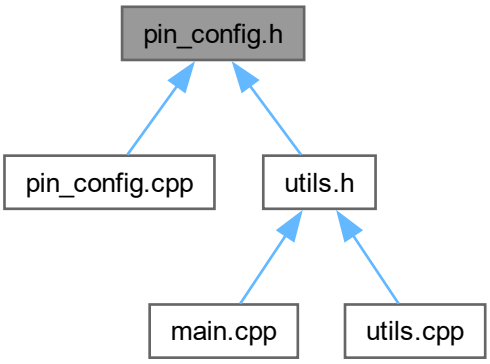
```
00001 #include "pin_config.h"
00002
00003 const int csPin = 1;    // LoRa radio chip select
00004 const int resetPin = 29; // LoRa radio reset
00005 const int irqPin = 10;     // change for your board; must be a hardware interrupt pin
```

## 3.5 pin_config.h File Reference

`#include <stdint.h>`
Include dependency graph for pin_config.h:




This graph shows which files directly or indirectly include this file:



**Macros**

- #define DEBUG_UART_PORT uart1
- #define BAUD_RATE 115200
- #define UART_TX_PIN 8
- #define UART_RX_PIN 9
- #define PIN_MISO 4
- #define PIN_CS 1
- #define PIN_SCK 2
- #define PIN_MOSI 3
- #define SPI_PORT spi0
- #define READ_BIT 0x80
- #define LORA_DEFAULT_SPI spi0
- #define LORA_DEFAULT_SPI_FREQUENCY 8E6
- #define LORA_DEFAULT_SS_PIN 1
- #define LORA_DEFAULT_RESET_PIN 29
- #define LORA_DEFAULT_DIO0_PIN 0
- #define PA_OUTPUT_RFO_PIN 11
- #define PA_OUTPUT_PA_BOOST_PIN 12

**Variables**

- const int csPin
- const int resetPin
- const int irqPin
- uint8_t localAddress

  *Address of this device (ground station)*
- uint8_t destination

  *Address of the destination device (satellite)*

## 3.5.1 Macro Definition Documentation

### 3.5.1.1 DEBUG_UART_PORT

```
#define DEBUG_UART_PORT uart1
```

Definition at line 7 of file pin_config.h.

### 3.5.1.2 BAUD_RATE

```
#define BAUD_RATE 115200
```

Definition at line 8 of file pin_config.h.

### 3.5.1.3 UART_TX_PIN

```
#define UART_TX_PIN 8
```

Definition at line 9 of file pin_config.h.

#### 3.5.1.4 UART_RX_PIN

```
#define UART_RX_PIN 9
```

Definition at line 10 of file pin_config.h.

#### 3.5.1.5 PIN_MISO

```
#define PIN_MISO 4
```

Definition at line 12 of file pin_config.h.

#### 3.5.1.6 PIN_CS

```
#define PIN_CS 1
```

Definition at line 13 of file pin_config.h.

#### 3.5.1.7 PIN_SCK

```
#define PIN_SCK 2
```

Definition at line 14 of file pin_config.h.

#### 3.5.1.8 PIN_MOSI

```
#define PIN_MOSI 3
```

Definition at line 15 of file pin_config.h.

#### 3.5.1.9 SPI_PORT

```
#define SPI_PORT spi0
```

Definition at line 17 of file pin_config.h.

#### 3.5.1.10 READ_BIT

```
#define READ_BIT 0x80
```

Definition at line 18 of file pin_config.h.

#### 3.5.1.11 LORA_DEFAULT_SPI

```
#define LORA_DEFAULT_SPI spi0
```

Definition at line 20 of file pin_config.h.

#### 3.5.1.12 LORA_DEFAULT_SPI_FREQUENCY

```
#define LORA_DEFAULT_SPI_FREQUENCY 8E6
```

Definition at line 21 of file pin_config.h.

#### 3.5.1.13 LORA_DEFAULT_SS_PIN

```
#define LORA_DEFAULT_SS_PIN 1
```

Definition at line 22 of file pin_config.h.

#### 3.5.1.14 LORA_DEFAULT_RESET_PIN

```
#define LORA_DEFAULT_RESET_PIN 29
```

Definition at line 23 of file pin_config.h.

#### 3.5.1.15 LORA_DEFAULT_DIO0_PIN

```
#define LORA_DEFAULT_DIO0_PIN 0
```

Definition at line 24 of file pin_config.h.

#### 3.5.1.16 PA_OUTPUT_RFO_PIN

```
#define PA_OUTPUT_RFO_PIN 11
```

Definition at line 26 of file pin_config.h.

#### 3.5.1.17 PA_OUTPUT_PA_BOOST_PIN

```
#define PA_OUTPUT_PA_BOOST_PIN 12
```

Definition at line 27 of file pin_config.h.

### 3.5.2 Variable Documentation

#### 3.5.2.1 csPin

```
const int csPin  [extern]
```

Definition at line 3 of file pin_config.cpp.

#### 3.5.2.2 resetPin

```
const int resetPin  [extern]
```

Definition at line 4 of file pin_config.cpp.

#### 3.5.2.3 irqPin

```
const int irqPin  [extern]
```

Definition at line 5 of file pin_config.cpp.

#### 3.5.2.4 localAddress

```
uint8_t localAddress  [extern]
```

Address of this device (ground station)

Definition at line 26 of file main.cpp.

#### 3.5.2.5 destination

```
uint8_t destination  [extern]
```

Address of the destination device (satellite)

Definition at line 31 of file main.cpp.

## 3.6 pin_config.h

Go to the documentation of this file.

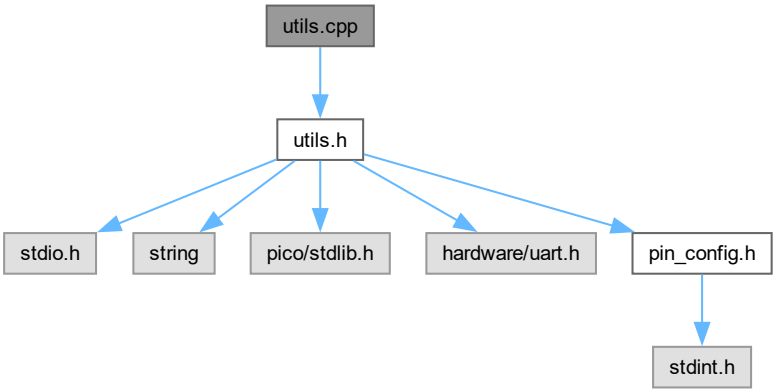
```
00001 // pin_config.h
00002 #include <stdint.h>
00003
00004 #ifndef PIN_CONFIG_H
00005 #define PIN_CONFIG_H
00006
00007 #define DEBUG_UART_PORT uart1
00008 #define BAUD_RATE 115200
00009 #define UART_TX_PIN 8
00010 #define UART_RX_PIN 9
00011
00012 #define PIN_MISO 4
00013 #define PIN_CS   1
00014 #define PIN_SCK  2
00015 #define PIN_MOSI 3
00016
00017 #define SPI_PORT spi0
00018 #define READ_BIT 0x80
00019
00020 #define LORA_DEFAULT_SPI           spi0
00021 #define LORA_DEFAULT_SPI_FREQUENCY 8E6
00022 #define LORA_DEFAULT_SS_PIN        1
00023 #define LORA_DEFAULT_RESET_PIN     29
00024 #define LORA_DEFAULT_DIO0_PIN      0
00025
00026 #define PA_OUTPUT_RFO_PIN          11
00027 #define PA_OUTPUT_PA_BOOST_PIN     12
00028
00029 // LoRa constants - declare as extern
00030 extern const int csPin;       // LoRa radio chip select
00031 extern const int resetPin;    // LoRa radio reset
00032 extern const int irqPin;      // LoRa hardware interrupt pin
00033 extern uint8_t localAddress; // address of this device
00034 extern uint8_t destination;  // destination to send to
00035
00036
00037 #endif // PIN_CONFIG_H
```

## 3.7 README.md File Reference

## 3.8 utils.cpp File Reference

```
#include "utils.h"
```

Include dependency graph for utils.cpp:




### Functions

- void uartPrint (const std::string msg, uart_inst_t *uart)

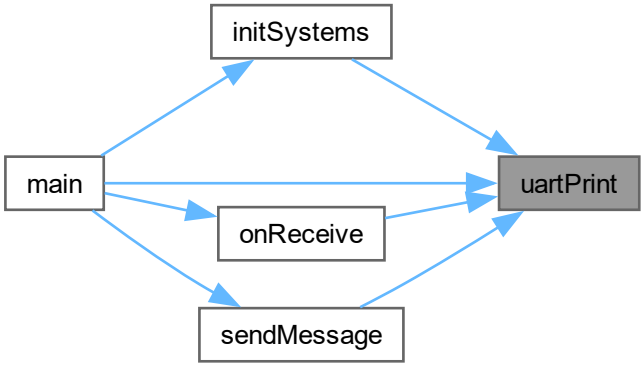
### 3.8.1 Function Documentation

#### 3.8.1.1 uartPrint()

```
void uartPrint (
            const std::string msg,
            uart_inst_t * uart)
```

Definition at line 3 of file utils.cpp.

Here is the caller graph for this function:




## 3.9 utils.cpp

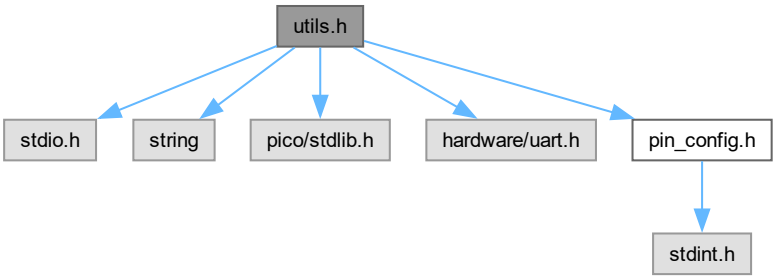
Go to the documentation of this file.

```
00001 #include "utils.h"
00002
00003 void uartPrint(const std::string msg, uart_inst_t* uart)
00004 {
00005     uint32_t timestamp = to_ms_since_boot(get_absolute_time());
00006     std::string msgToSend = "[" + std::to_string(timestamp) + "ms] - " + msg;
00007     uart_puts(uart, msgToSend.c_str());
00008     uart_puts(uart, "\r\n");
00009 }
```

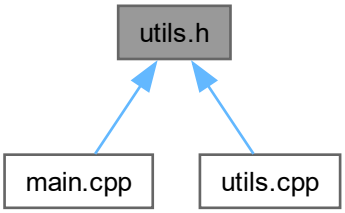
## 3.10 utils.h File Reference

```
#include <stdio.h>
#include <string>
#include "pico/stdlib.h"
#include "hardware/uart.h"
#include "pin_config.h"
```

Include dependency graph for utils.h:



This graph shows which files directly or indirectly include this file:




**Functions**

- void uartPrint (const std::string msg, uart_inst_t *uart=DEBUG_UART_PORT)

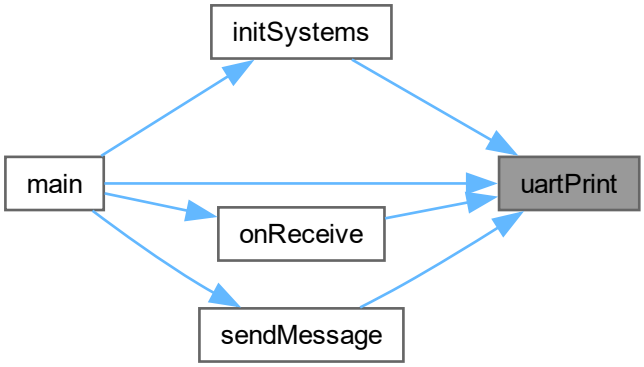
## 3.10.1 Function Documentation

### 3.10.1.1 uartPrint()

```
void uartPrint (
            const std::string msg,
            uart_inst_t * uart = DEBUG_UART_PORT)
```

Definition at line 3 of file utils.cpp.

Here is the caller graph for this function:



# 3.11 utils.h

Go to the documentation of this file.

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003
00004 #include <stdio.h>
00005 #include <string>
00006 #include "pico/stdlib.h"
00007 #include "hardware/uart.h"
00008 #include "pin_config.h"
00009
00010 void uartPrint(const std::string msg, uart_inst_t* uart = DEBUG_UART_PORT);
00011
00012 #endif
```

# Index


BAUD_RATE
  pin_config.h, 14

csPin
  pin_config.cpp, 12
  pin_config.h, 16

DEBUG_UART_PORT
  pin_config.h, 14
destination
  main.cpp, 9
  pin_config.h, 17

initSystems
  main.cpp, 8
irqPin
  pin_config.cpp, 12
  pin_config.h, 17

localAddress
  main.cpp, 9
  pin_config.h, 17
LORA_DEFAULT_DIO0_PIN
  pin_config.h, 16
LORA_DEFAULT_RESET_PIN
  pin_config.h, 16
LORA_DEFAULT_SPI
  pin_config.h, 15
LORA_DEFAULT_SPI_FREQUENCY
  pin_config.h, 15
LORA_DEFAULT_SS_PIN
  pin_config.h, 16

main
  main.cpp, 8
main.cpp, 5
  destination, 9
  initSystems, 8
  localAddress, 9
  main, 8
  onReceive, 6
  REPLY_TIMEOUT, 9
  sendMessage, 7

onReceive
  main.cpp, 6

PA_OUTPUT_PA_BOOST_PIN
  pin_config.h, 16
PA_OUTPUT_RFO_PIN
  pin_config.h, 16

pico-lora, 1
pin_config.cpp, 12
  csPin, 12
  irqPin, 12
  resetPin, 12
pin_config.h, 13
  BAUD_RATE, 14
  csPin, 16
  DEBUG_UART_PORT, 14
  destination, 17
  irqPin, 17
  localAddress, 17
  LORA_DEFAULT_DIO0_PIN, 16
  LORA_DEFAULT_RESET_PIN, 16
  LORA_DEFAULT_SPI, 15
  LORA_DEFAULT_SPI_FREQUENCY, 15
  LORA_DEFAULT_SS_PIN, 16
  PA_OUTPUT_PA_BOOST_PIN, 16
  PA_OUTPUT_RFO_PIN, 16
  PIN_CS, 15
  PIN_MISO, 15
  PIN_MOSI, 15
  PIN_SCK, 15
  READ_BIT, 15
  resetPin, 16
  SPI_PORT, 15
  UART_RX_PIN, 14
  UART_TX_PIN, 14
PIN_CS
  pin_config.h, 15
PIN_MISO
  pin_config.h, 15
PIN_MOSI
  pin_config.h, 15
PIN_SCK
  pin_config.h, 15

READ_BIT
  pin_config.h, 15
README.md, 18
REPLY_TIMEOUT
  main.cpp, 9
resetPin
  pin_config.cpp, 12
  pin_config.h, 16

sendMessage
  main.cpp, 7
SPI_PORT
  pin_config.h, 15

UART_RX_PIN
    pin_config.h, 14
UART_TX_PIN
    pin_config.h, 14
uartPrint
    utils.cpp, 18
    utils.h, 20
utils.cpp, 18
    uartPrint, 18
utils.h, 19
    uartPrint, 20